

Arrays, Types, Objetos, Interfaces e Classes

Miguel Costa
miguelangcosta@gmail.com

Objetivos

- Aprender a usar **arrays tipados**
- Introduzir **interfaces** para organizar objetos
- Explorar **Union Types** e variáveis opcionais (?)
- Preparar para **classes e objetos** na próxima aula
- Continuar aplicando conceitos no **gestor de tarefas**

Union Types

Union Types permitem que uma variável aceite **mais de um tipo**, útil quando o valor pode variar entre tipos específicos, como `string` ou `number`.

```
let id: number | string;

id = 123;      // OK
id = "abc123"; // OK
// id = true;  // ✗ Erro: boolean não é permitido
```

Dica: Usar Union Types quando a variável pode ter valores diferentes, mas limitados a tipos conhecidos.

Vantagens de Union Types

Flexibilidade sem perder tipagem!

Permite lidar com dados que **realmente podem variar**, sem recorrer a **any**.

```
function imprimir(id: number | string) {  
  console.log(id);  
}
```

Melhor do que:

```
function imprimir(id: any) { ... } // perde segurança
```

Vantagens de Union Types

Modelar melhor dados do mundo real!

APIs, formulários e bases de dados frequentemente retornam valores diferentes:

```
type ApiResponse = string | null;  
  
let resposta: ApiResponse;
```

Exemplos comuns:

- string | null
- number | undefined

Vantagens de Union Types

Melhor reutilização de código

Uma função pode trabalhar com vários cenários:

```
type ID = number | string | null;

function showId(id: ID) {
  if (id === null) {
    console.log("No ID provided");
  } else {
    console.log(`ID: ${id}`);
  }
}

showId(10);      // ID: 10
showId("xyz");   // ID: xyz
showId(null);    // No ID provided
```

Union Types como usar?

Quando faz sentido usar?

- ✓ Dados vindos de APIs
- ✓ Inputs de utilizador
- ✓ Estados e enums tipados
- ✓ Funções genéricas
- ✓ Migração gradual de JS → TS

Quando evitar?

- ✗ Quando o tipo é **sempre o mesmo**
- ✗ Como substituto de **any**
- ✗ Quando um **objeto ou interface** resolveria melhor

Arrays tipados

Em TypeScript, podemos declarar **arrays com tipos específicos**.

```
let tarefas: string[] = ["Comprar pão", "Estudar TS"];  
let numeros: number[] = [1, 2, 3];
```

- Um array de **string** não pode conter números
- Um array de **number** não pode conter strings

Boas práticas: sempre tipar arrays para que o TypeScript detecte erros cedo.

Arrays com Union

Podemos ter arrays que aceitam **mais de um tipo de valor** usando |:

```
let ids: (string | number)[] = [1, "abc", 3];
```

Cada elemento pode ser **string ou number**

Útil para listas de IDs, inputs variados ou valores opcionais

O que são dados?

Dados são informação usada pela aplicação.

Exemplo (tarefa):

- texto
- prioridade
- Concluída

```
const texto = "Estudar TypeScript";  
const prioridade = "alta";  
const concluida = false;
```

Objeto

Um objeto serve para guardar dados rapidamente.

Um objeto agrupa dados relacionados.

```
const tarefa = {  
  texto: "Estudar TypeScript",  
  prioridade: "alta",  
  concluida: false  
};
```

- Simples e direto
- Ideal para dados vindos do formulário

type

`type` define como um objeto deve ser.

```
type Tarefa = {  
  texto: string;  
  prioridade: string;  
  concluida: boolean;  
};
```

Uso:

```
const tarefa: Tarefa = {  
  texto: "Fazer exercícios",  
  prioridade: "media",  
  concluida: false  
};
```

Interfaces

O que são interfaces?

- Uma **interface** é como um **molde para objetos**.
- Define **quais propriedades e tipos** um objeto deve ter, mas **não cria código executável**.
- Serve para garantir que objetos sigam uma estrutura consistente.

```
interface Tarefa {  
  titulo: string;  
  prioridade: "alta" | "media" | "baixa";  
  concluida?: boolean; // ? = opcional  
}  
  
let tarefa1: Tarefa = {  
  titulo: "Estudar TypeScript",  
  prioridade: "alta",  
};
```

Interfaces são **tipo “contratos”**.

Não geram código JS, apenas ajudam o TypeScript a verificar tipos.

Interfaces

- Uma interface **não contém implementação**, mas pode **definir que um objeto terá funções**.
- Apenas declara a assinatura da função, **sem corpo**.

```
interface Tarefa {  
  id: number;  
  titulo: string;  
  concluida: boolean;  
  marcarConcluida(): void; // apenas assinatura  
}
```

A interface define **como o objeto deve ser**, mas **não cria comportamento sozinho**.

```
const tarefa: Tarefa = {  
  id: 1,  
  titulo: "Estudar TS",  
  concluida: false,  
  marcarConcluida() {  
    this.concluida = true;  
  }  
};
```

Um objeto que siga a interface deve **implementar essa função**

type vs interface

`type` é o mesmo que `interface`?

👉 Quase, mas não exatamente.

O que têm em comum

- Servem para **descrever a forma dos dados**
- Só existem em TypeScript
- Desaparecem no JavaScript compilado
- Não criam objetos
- Não geram código em runtime

```
type Task = {  
  title: string;  
  done: boolean;  
};  
  
interface TaskI {  
  title: string;  
  done: boolean;  
}
```

type vs interface

Então qual é a diferença? **interface** pode ser estendida automaticamente (merge)

Resultado:

```
interface User {  
  name: string;  
}
```

```
interface User {  
  age: number;  
}
```

```
User = {  
  name: string;  
  age: number;  
}
```

- Isto chama-se **declaration merging**
- Muito usado em bibliotecas e frameworks

```
interface User {  
  name: string;  
}
```

```
interface User {  
  name: number; // ✗ erro: 'name' must be of type 'string'
```

Redefinição de propriedades na interface não funciona!

type vs interface

type NÃO permite merge

```
type User = {  
  name: string;  
};
```

```
type User = {  
  age: number;  
}; // ✗ ERRO
```

type vs interface

type é mais poderoso

type pode:

- Criar **union types**
- Criar **intersection types**
- Tipar primitivos, arrays, funções

type vs interface

type Union (|)

```
type ID = string | number;

type Task = {
  id: ID;
};
```

interface não consegue fazer isto:

```
interface ID = string | number; // ❌ impossível
```

type Intersection (&)

```
type WithId = {
  id: number;
};

type WithName = {
  name: string;
};

type User = WithId & WithName;
```

Resultado:

```
User = {
  id: number;
  name: string;
}
```

type vs interface

Regra simples para iniciantes: **type** junta tipos, **interface** cresce com o projeto.

Usa **type** por defeito

- Mais simples
- Mais flexível
- Tens union (|)
- Tens intersection (&)

Usa **interface** quando:

- Trabalhas com **classes**
- Estás a criar **APIs públicas**
- Vais usar **implements**
- Precisas de merge

Classes

É um **molde** ou **modelo** que define:

- **Propriedades** (dados)
- **Métodos** (funções/comportamentos)
- Não ocupa memória até que seja criada uma instância.
- Serve para criar **vários objetos com a mesma estrutura e comportamento**.

```
class Tarefa {  
  id: number;  
  titulo: string;  
  concluida: boolean = false;  
  
  constructor(id: number, titulo: string) {  
    this.id = id;  
    this.titulo = titulo;  
  }  
  
  marcarConcluida() {  
    this.concluida = true;  
  }  
}
```

implements: ligar classe à interface

interface: define o contrato

```
interface TarefaInterface {  
    texto: string;  
    prioridade: string;  
    concluida: boolean;  
}
```

class: é obrigada a seguir:

```
class Tarefa implements TarefaInterface {  
    texto: string;  
    prioridade: string;  
    concluida: boolean;  
  
    constructor(texto: string, prioridade: string) {  
        this.texto = texto;  
        this.prioridade = prioridade;  
        this.concluida = false;  
    }  
}
```

implements: Múltiplas Interfaces

Uma **class** pode implementar várias interfaces

```
interface A {  
    value: string | number;  
}  
  
interface B {  
    value: number;  
}  
  
class MyClass implements A, B {  
    value: number; // ✅ ok, number é compatível com string | number  
}
```

implements: Múltiplas Interfaces

Os tipos das múltiplas interfaces têm de ser compatíveis:

```
interface A {  
  value: string;  
}  
  
interface B {  
  value: number;  
}  
  
class MyClass implements A, B {  
  value: ??? // ✗ TypeScript não sabe qual tipo usar  
}
```


Fluxo recomendado

1. Define os dados com:
 - `type` ou `interface`
2. Usa objetos para guardar dados
3. Introduz classes **apenas quando há comportamento**
4. Usa `implements` para garantir estrutura

Simplificado:

Objetos → dados

`type` → forma dos dados (flexível)

`interface` → contrato de objetos

Classe → dados + comportamento

`implements` → classe respeita o contrato

“Se não há lógica, não há classe.”

Variáveis opcionais (?)

- Marcar propriedades como **opcionais** permite flexibilidade
- Útil quando nem todo objeto precisa ter todos os campos

```
interface Tarefa {  
  titulo: string;  
  descricao?: string;  
}  
  
let tarefa2: Tarefa = { titulo: "Aprender TS" };
```

`descricao` pode estar presente ou não

Evita erro de "propriedade não existe"

Optional chaining (?.)

1) Acessar propriedade de objeto opcional

```
const user = { name: "Ana" };

console.log(user.address?.city); // undefined (sem erro)
```

2) Acessar método que pode não existir

```
const logger = {
  log: (msg: string) => console.log(msg),
};

logger.log?.("Hello"); // funciona
logger.error?.("Oops"); // não existe, não dá erro
```

3) Acessar array opcional

```
const data: { items?: string[] } = {};

console.log(data.items?.[0]); // undefined (sem erro)
```

Nullish coalescing (??)

1) Valor padrão quando for null ou undefined

```
const username = null;

const displayName = username ?? "Visitante";

console.log(displayName); // "Visitante"
```

2) Diferença entre || e ??:

- || (ou) considera como false:
- 0
- "" (string vazia)
- false
- null
- undefined

```
const count = 0;

console.log(count || 10); // 10 (0 é falsy)
console.log(count ?? 10); // 0 (0 é válido)
```

3) Usar com optional chaining

```
const user: any = {};

const city = user.address?.city ?? "Cidade não informada";

console.log(city); // "Cidade não informada"
```

Questões